

Bilinear by Default: A Criterion for Koopman Rollouts in Sampling-Based Predictive Control

Abstract—The rollout model is the central design choice in sampling-based predictive control: Model Predictive Path Integral (MPPI) control is derivative-free and massively parallel, but only as good as the model through which it rolls out trajectories. Koopman operator models are attractive here, lifting nonlinear dynamics to an almost-linear evolution whose rollouts are inexpensive matrix–vector products with data-driven error bounds. Existing Koopman-MPPI keeps the lifted model *linear in the input* and succeeds on many systems, yet no criterion says when that linear choice is safe. This paper supplies one. We show that a bilinear lift is needed when the input reaches the tracked output through a *configuration-dependent* gain that varies appreciably over the operating region; under the standard linear-readout realization this is an input–state coupling that a constant input matrix cannot match, however rich the dictionary, since enriching it lifts only the state. Since sampling-based control imposes no convexity, the *bilinear* model—the correct minimal class for control-affine systems—is then the right default rollout at the same matrix–vector cost, made error-aware by an online error-tube cost with an accompanying free-energy stability analysis. Across four benchmarks, where the gain varies maximally—a wheeled vehicle and an aerial drone commanded in body-frame velocity—bilinear is decisive: it tracks at oracle accuracy where linear deep-Koopman MPPI fails by more than two orders of magnitude in prediction and structurally in control. Where the gain barely varies—acceleration- or force-driven systems—the two coincide, which explains prior linear-Koopman successes. Full hyperparameters and videos: <https://koopman-mppi-anonymous.github.io>.

Index Terms—Model predictive path integral control, Koopman operator, bilinear systems, sampling-based MPC, optimization and optimal control, robot learning.

I. INTRODUCTION

SAMPLING-BASED model predictive control, most prominently Model Predictive Path Integral (MPPI) control [1]–[3], has become a workhorse for agile robotics. It is derivative-free, embarrassingly parallel on GPUs, and, through its information-theoretic derivation [4], [5], places essentially no structural requirement on the dynamics it rolls out. This generality comes at a price that is easy to overlook: at every control step the controller propagates thousands of sampled trajectories over a horizon, so the *rollout model* is both the dominant computational cost and the single most important design choice [6].

The Koopman operator offers a principled way to make this trade favorable. By lifting the state through a dictionary of observables [7]–[9], the nonlinear dynamics become (near-)linear in the lifted space, so a rollout reduces to a sequence of matrix–vector products that are orders of magnitude cheaper than integrating the original dynamics, are trivially parallelized, and, where no analytical model is available, identified directly from data [10], [11]. Recent work couples this idea with MPPI and reports substantial speedups using a learned

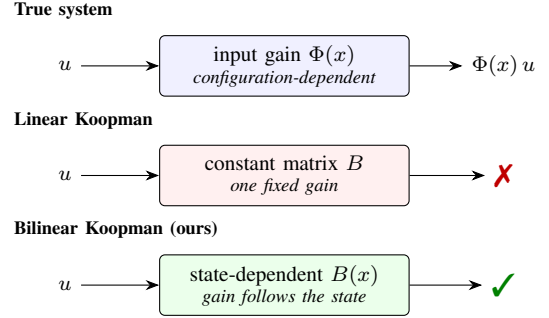


Fig. 1. Why the lift may have to be bilinear. The command u reaches the tracked output through a *configuration-dependent* gain $\Phi(x)$, such as the heading rotation of a vehicle or a body-to-world rotation of a drone. A *linear* Koopman lift can apply only a *constant* input matrix, so it cannot reproduce $\Phi(x)$ when the configuration varies (X); enriching the state dictionary does not help, because the input still enters through a constant matrix. A *bilinear* lift lets the input gain follow the state and matches $\Phi(x)$ (✓), at the same rollout cost.

linear Koopman model [12], while bilinear Koopman models have proven effective for control within (nonconvex) model predictive control [13]–[16]. Throughout, we assume the analytical dynamics are unknown or too costly to integrate at the sampling rate (the standard model-free, real-time setting), so a true-dynamics rollout is available only as an *oracle* reference, not as a deployable controller.

The choice between a linear and a bilinear lift, however, has been made by convention rather than by criterion. A linear Koopman model lets the input act through a single fixed map, the same in every configuration; a bilinear one lets the input gain follow the state (Fig. 1). Linear-predictor Koopman MPC keeps the program convex by construction [17], [18], so it must stay linear; sampling-based MPPI carries no such requirement, so neither restriction is forced. What has been missing is an answer to the underlying question: *when does a Koopman rollout actually have to be bilinear?*

We answer it. Our criterion is a property of the system, not of the optimizer: a bilinear lift is needed when the input reaches the tracked output through a state-dependent gain $\Phi(x)$ that varies appreciably over the operating region. Crucially, under the standard Koopman-MPPI realization—a linear readout with a constant input matrix—this input–state coupling cannot be matched to first order: enriching the dictionary lifts the *state*, but the input still multiplies a *constant* matrix, so a state-dependent gain induces an irreducible error that a deep linear lift incurs just as a fixed one does.¹ Bilinearity is then a structural requirement, not a matter of capacity.

¹ A nonlinear readout, or an input-history (time-delay) lift, could in principle mitigate this; we analyze the standard linear-readout realization used by linear Koopman-MPPI.

Because sampling removes the convexity that forced the linear choice, we adopt the bilinear model (the correct minimal class for control-affine systems) as the default rollout at the same matrix–vector cost, and make it model-error-aware through an online error-tube cost [18], [19] with an accompanying free-energy stability analysis.

Our contributions are:

- **A criterion for the lift.** We identify when a Koopman rollout must be bilinear—when the input reaches the tracked output through a configuration-dependent gain that varies over the operating region—formalized as Prop. 1 (Sec. III).
- **No lift can substitute.** Under a linear readout with a constant input matrix, this coupling cannot be absorbed by enriching the state dictionary, so a deep linear lift incurs the same irreducible error as a fixed one; bilinearity is structural, not capacity.
- **Bilinear is the free default.** Because sampling imposes no convexity, the bilinear model serves as the default MPPI rollout at the same matrix–vector cost, made error-aware by a tube cost; for certified identifiers this yields a free-energy practical-stability guarantee, for which our deep implementation uses an empirical proxy.
- **A map that subsumes prior work.** The criterion explains where linear Koopman-MPPI succeeds (acceleration/force-driven, low-variation systems) and where it must fail (velocity-commanded mobile bases), placing prior linear and bilinear Koopman results on a single axis (Sec. IV, V).

II. PROBLEM STATEMENT AND BACKGROUND

System and objective. We consider a control-affine system $\dot{x} = f(x) + G(x)u$, $x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$, discretized as

$$x_{k+1} = F(x_k, u_k), \quad (1)$$

known only through input–state data $\{(x_k, u_k, x_{k+1})\}$. Over a horizon T , the controller seeks the input sequence $U = (u_0, \dots, u_{T-1})$ minimizing the expected cost

$$J(U) = \mathbb{E} \left[\phi(x_T) + \sum_{k=0}^{T-1} q(x_k, u_k) \right]. \quad (2)$$

Information-theoretic MPPI. MPPI draws K input sequences $V^{(k)} = U + \mathcal{E}^{(k)}$ with $\varepsilon_t^{(k)} \sim \mathcal{N}(0, \Sigma)$, rolls each through the dynamics to obtain a trajectory cost $S(V^{(k)})$, and updates each input by the cost-weighted average of the perturbations,

$$u_t \leftarrow u_t + \frac{\sum_k w^{(k)} \varepsilon_t^{(k)}}{\sum_k w^{(k)}}, \quad (3)$$

$$w^{(k)} = \exp \left(-\frac{1}{\lambda} (S(V^{(k)}) - \rho) \right),$$

with temperature $\lambda > 0$ and $\rho = \min_k S(V^{(k)})$. The derivation places no convexity, differentiability, or affine-in-input requirement on F [3]; the only step that scales with the sample count is the per-sample rollout evaluating $S(V^{(k)})$.

Koopman lifting and identification. Koopman methods lift the state through a dictionary $\Psi : \mathbb{R}^n \rightarrow \mathbb{R}^N$ with $\Psi(0) = 0$

and a left inverse C ($x = C\Psi(x)$), and model the evolution of $z = \Psi(x)$. The *linear* and *bilinear* predictors are

$$(\text{linear}) \quad z^+ \approx Az + Bu, \quad (4)$$

$$(\text{bilinear}) \quad z^+ \approx Az + B_0u + \sum_{i=1}^m u_i (B_i z). \quad (5)$$

Because the input enters the lifted dynamics through state-dependent vector fields, a finite *linear* predictor is generically inexact, whereas the *bilinear* form (5) is the provably correct minimal class [16]. Both are identified by (regularized) least squares, using extended dynamic mode decomposition with control [9], [10] for a fixed dictionary or jointly with a learned deep dictionary [11], so that (5) simply augments the regressor with the products $u_i z$. Writing $M(u) := A + \sum_i u_i B_i$, one bilinear step is $z^+ = M(u)z + B_0u$: an input-dependent matrix, but still $m+1$ matrix–vector products, the same rollout regime as the linear model (4).

Proportional error bound. Let $r(x, u)$ be the one-step residual of (5). Certified Koopman identification [18], [19] provides a *proportional* bound

$$\|r(x, u)\| \leq c_x \|\Psi(x)\| + c_u \|u\|, \quad (6)$$

whose constants c_x, c_u are estimated from data and *vanish at the equilibrium*; Sec. III propagates it into the sampling cost.

III. METHOD: A CRITERION, AND BILINEAR KOOPMAN MPPI

Our controller is standard information-theoretic MPPI (3), but every rollout is performed in the lifted space by the *bilinear* surrogate (5). At each step we lift the measured state, roll K sampled input sequences through $\hat{z}^+ = M(v)\hat{z} + B_0v$, score them with the task cost plus an error-tube penalty, and apply the softmax-weighted update (Algorithm 1). Each rollout step is one $N \times N$ product for A , m for the $\{B_i\}$, and the affine term B_0u : an $m+1$ factor over the single product of a linear model, a modest constant that fuses into dense batched matrix multiplications on a GPU, so wall-clock cost is set by the lifted dimension N , not by the bilinear structure.

Training. One-step regression of (5) yields poor multi-step rollouts; we instead minimize a *multi-step* loss over length- H windows,

$$\mathcal{L} = \sum_{t=1}^H \|C(\hat{z}_t - z_t)\|^2 + \gamma \|\hat{z}_t - \Psi(x_t)\|^2, \quad (7)$$

rolling $\hat{z}_t$ from $\hat{z}_0 = \Psi(x_0)$ through (5), with the second term enforcing latent consistency. For the multi-input systems ($m \geq 3$) we initialize $\{B_i\}$ to zero, so training starts from the linear model and improves on it (for $m \leq 2$ a small random initialization is equally stable), and, for systems deployed under sustained maneuvers, draw coherent control sequences matching that distribution.

When a bilinear lift is required. Let $y = h(x) = Cz$ be the tracked output. To first order in the step, the control-affine dynamics give

$$y_{k+1} - y_k = \underbrace{\nabla h(x_k) f(x_k) \Delta t}_{d(x_k)} + \underbrace{\nabla h(x_k) G(x_k) \Delta t}_{\Phi(x_k)} u_k + O(\Delta t^2), \quad (8)$$

Algorithm 1 Bilinear Koopman MPPI (one control step)

Require: state x ; warm start U ; model $A, B_0, \{B_i\}, C$; bound c_x, c_u ; weights λ, β

- 1: $z_0 \leftarrow \Psi(x)$
- 2: **for** $k = 1, \dots, K$ **in parallel do**
- 3: $V^{(k)} \leftarrow U + \mathcal{E}^{(k)}, \quad \varepsilon_t^{(k)} \sim \mathcal{N}(0, \Sigma)$
- 4: $\hat{z} \leftarrow z_0, \quad e \leftarrow 0, \quad S^{(k)} \leftarrow 0$
- 5: **for** $t = 0, \dots, T - 1$ **do**
- 6: $\hat{z} \leftarrow M(v_t^{(k)}) \hat{z} + B_0 v_t^{(k)}$
- 7: $e \leftarrow (\|M(v_t^{(k)})\| + c_x)e + c_x \|\hat{z}\| + c_u \|v_t^{(k)}\|$
- 8: $S^{(k)} \leftarrow S^{(k)} + q(C\hat{z}, v_t^{(k)}) + \beta e$
- 9: **end for**
- 10: $S^{(k)} \leftarrow S^{(k)} + \phi(C\hat{z})$
- 11: **end for**
- 12: $w^{(k)} \leftarrow \exp(-\frac{1}{\lambda}(S^{(k)} - \min_j S^{(j)}))$
- 13: $U \leftarrow U + (\sum_k w^{(k)} \mathcal{E}^{(k)}) / \sum_k w^{(k)}$
- 14: **return** u_0 ; shift U for the next step

so the input enters y through the state-dependent gain $\Phi(x)$.

Proposition 1 (A constant input matrix cannot encode a state-dependent gain). *Let the tracked output evolve, to first order, as $y_{k+1} = y_k + \Phi(x_k)u_k + d(x_k)$. A linear Koopman model (4) reproduces this input response for all (x, u) only if Φ is constant; if Φ varies with the configuration, the linear model's output error is of the order of the variation of Φ over the operating set, whereas the bilinear model (5) reproduces $\Phi(x)u$ exactly whenever the dictionary spans the entries of Φ .*

Proof. For (4), $\partial y_{k+1} / \partial u = CB$ is constant, so matching $\Phi(x)$ for all x forces $\Phi \equiv CB$. For (5), $\partial y_{k+1} / \partial u_i = C(B_0 e_i + B_i z)$ is affine in the lifted state, so $\Phi(x)u$ is reproduced once each entry of $\Phi(x)$ lies in $\text{span } \Psi$. $\square$

Remark 1 (a richer lift cannot substitute). The coupling $\Phi(x)u$ is a product of the input with a state-dependent gain. A Koopman lift enriches the representation of the *state*, but in the standard realization—a constant input matrix with a linear readout—the input still multiplies a fixed B ; no choice of state dictionary, however expressive (including a deep, learned one), makes a constant B reproduce a state-dependent $\Phi(x)$. Hence when Φ varies appreciably an irreducible constant- B error persists regardless of lift richness, and the bilinear term $u_i(B_i z)$ is what recovers it. This is distinct from the capacity role of richer lifts on weakly coupled systems (Sec. IV); realizations that break the standard form—a nonlinear readout or an input-history lift—lie outside this analysis.

Remark 2 (what makes Φ vary). The decisive quantity is the variation of Φ over the operating set, and a common driver is how the input enters. For a *velocity*-commanded mobile base the body-frame command reaches the world-frame output through a heading rotation, $\Phi(x) = R(\theta)$, which sweeps its full range as the robot turns—maximal variation, so bilinearity is essential. When the input acts at *second* order, through acceleration or force, $\Phi(x)u$ reaches y only after a further integration and its variation is diluted by $O(\Delta t^2)$, so a linear lift typically suffices; we test both regimes in Sec. IV.

Examples. For a unicycle (output the planar position)

$\Phi(x) \propto (\cos \theta, \sin \theta)$ rotates with the heading; for a body-velocity drone (output the world position) Φ is the body-to-world rotation; both sweep their full range, so a constant input matrix is correct at a single configuration only and the linear model fails outright.

Error-tube cost. The surrogate is inexact, so we discount samples whose *certified* error is large. Propagating the proportional bound (6) along a rollout gives the scalar tube of Algorithm 1 (line 7), with $e_0 = 0$,

$$e_{t+1} = (\|M(v_t)\| + c_x)e_t + c_x \|\hat{z}_t\| + c_u \|v_t\|, \quad (9)$$

and we add $\beta \sum_t e_t$ to each sample cost. Because (6) vanishes at the equilibrium, this penalty regularizes the search toward the certified region; $\beta \rightarrow 0$ recovers plain bilinear MPPI.

Proposition 2 (Tube bound). *Along any shared input sequence, the tube (9) satisfies $\|z_t - \hat{z}_t\| \leq e_t$ for all t , hence $\|x_t - \hat{x}_t\| \leq \|C\| e_t$.*

Proof. By induction: $e_0 = 0$ matches $z_0 = \hat{z}_0$; assuming $\|z_t - \hat{z}_t\| \leq e_t$, subtract the true and nominal updates, take norms, and bound the residual by (6) using $\|z_t\| \leq \|\hat{z}_t\| + e_t$; all coefficients are nonnegative. $\square$

Stability. Let $\mathcal{F}_\bullet(x) = -\lambda \log \mathbb{E}_{V \sim p}[\exp(-S_\bullet(V; x)/\lambda)]$ be the free energy of the real and surrogate costs.

Theorem 1 (Free-energy practical stability). *If the surrogate is cost-controllable (from every state some admissible input sequence strictly decreases the surrogate cost, cf. [18]) and the tracked cost is Lipschitz, the model-induced free-energy gap obeys $|\mathcal{F}_{\text{real}} - \mathcal{F}_{\text{nom}}| \leq \Delta S_{\text{max}}$, the tube cost of Prop. 2; consequently the real closed loop is practically asymptotically stable with ultimate bound $R = O(c_x, c_u, \lambda, K^{-1/2})$, vanishing as the model and sampling errors vanish.*

Proof sketch. The gap follows from $|S_{\text{real}} - S_{\text{nom}}| \leq \Delta S_{\text{max}}$ (Prop. 2, cost-Lipschitzness) and monotonicity of $\log \sum \exp$; combined with the relaxed-dynamic-programming decrease of the surrogate value [17], [18] it gives the ultimate bound, needing only cost-Lipschitzness, not value smoothness. $\square$

Scope. The guarantee presumes *certified* constants c_x, c_u (e.g. from SafEDMD-grade identification); the deep dictionaries in our experiments do not yet furnish them, so Sec. IV substitutes fitted constants as an empirical proxy and the resulting claim is empirical rather than certified.

IV. EXPERIMENTS

Setup. The three controllers share a deep Koopman lifting (identical network width, training data, and multi-step loss) and differ only in the lifted model: *Ours* uses the bilinear form (5) and the *linear* baseline [12] the linear form (4). For a fair comparison the two learned controllers share the horizon, sample count, lifting dimension, and training budget of Table I; where a setting is method-specific we adopt the linear baseline's own reported choice. *Oracle* MPPI, rolling out the true dynamics, is a model-fidelity reference, not a performance ceiling: sampling-based MPPI is suboptimal even with the true model. The unicycle and drone use standard kinematic models (the drone at the body-velocity navigation level) and

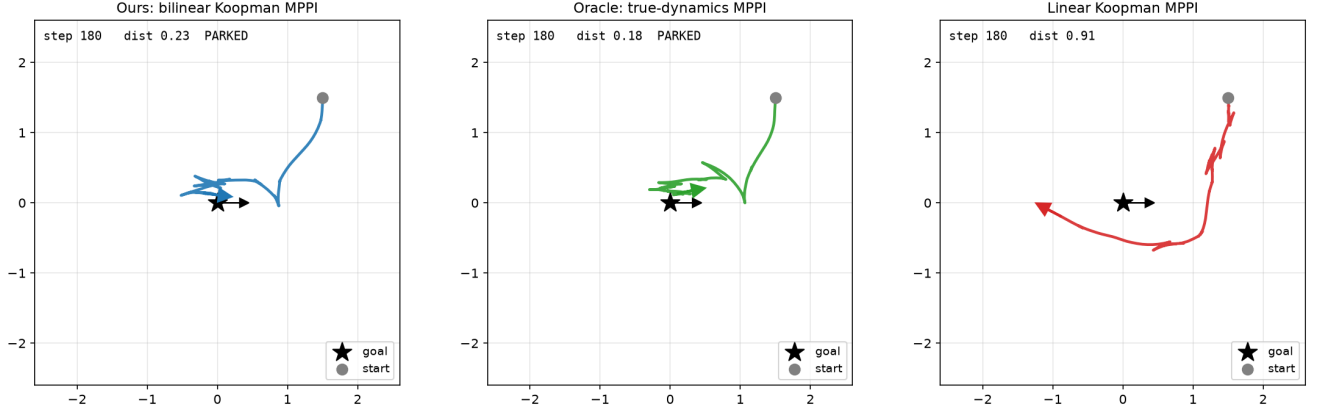


Fig. 2. Nonholonomic unicycle parking (start $\bullet$, goal $\star$; deep Koopman, representative run). Ours (bilinear, left) reaches the goal pose, matching the true-dynamics oracle (centre); the linear model (right) drives off, since a constant input matrix cannot represent the heading coupling $v(\cos \theta, \sin \theta)$. Parking rates over 20 trials: 95/85/0%.

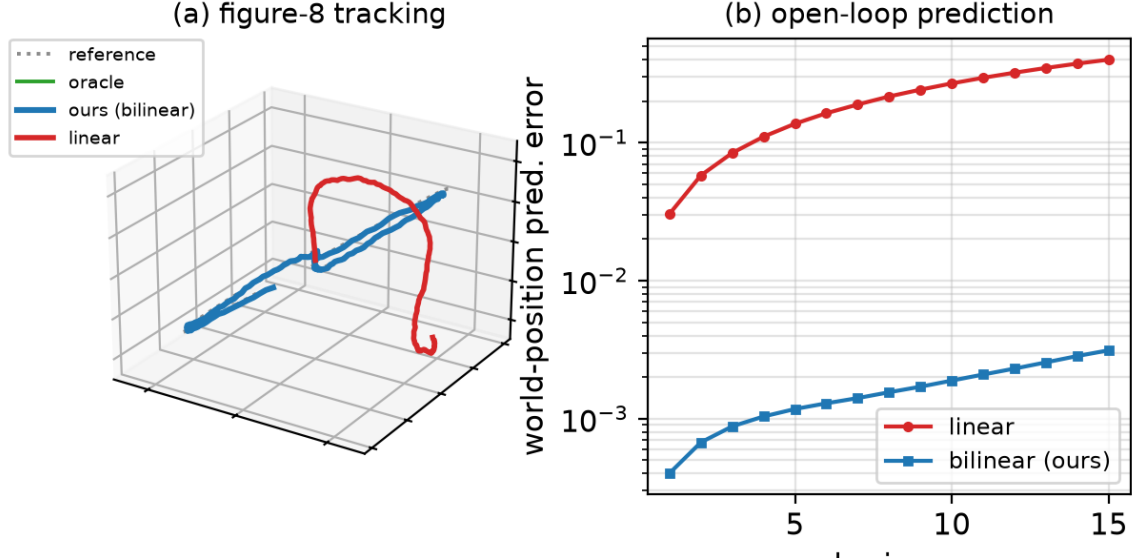


Fig. 3. 3D velocity-commanded drone. (a) Tracking a figure-8 while yawing along the path: ours (bilinear) coincides with the oracle, while the linear model drifts off because a constant input matrix cannot rotate the body-frame velocity command into the world frame. (b) Open-loop world-position prediction: the bilinear model is one-to-two orders of magnitude more accurate, exceeding $100\times$ at the planning horizon.

TABLE I
KEY PER-SYSTEM HYPERPARAMETERS.

System	N	K	T	Δt	λ
Unicycle	8	1536	40	0.05	0.4
Drone (3D)	13	512	15	0.05	0.25
Pendulum	8	2560	65	0.05	0.8
Quadrotor	24	1500	20	0.02	0.5

N , lifted dimension; K , samples; T , horizon; Δt , step size; λ , temperature.

the pendulum an analytic model; the quadrotor and the torque-controlled arm are simulated in MuJoCo [20], and the soft continuum arm in SoftManiSim [21]. Full settings and videos are on the project page: <https://koopman-mppi-anonymous.github.io>.

A. Where bilinear is decisive: a strongly varying input gain

Nonholonomic vehicle. A unicycle must park its pose at a goal; the forward-speed input enters the position through the heading, a configuration-dependent rotation that sweeps the full circle (Prop. 1). Rolled out over the horizon, the bilinear model predicts the position $42\text{--}48\times$ more accurately than the linear one (a fixed trigonometric dictionary gives the same picture, $37\text{--}46\times$). In closed loop (Fig. 2), ours parks reliably: across five training seeds and four start-goal pairs (20 trials) it reaches the goal in 95% of trials, matching the true-dynamics oracle (85%), whereas the linear model parks in 0%, driving off along its initial heading and ending 3.0 ± 1.9 m away.

Aerial drone. A quadrotor commanded by body-frame velocities (forward, lateral, vertical, and yaw rate) must track a 3D reference; the world position integrates the body velocity through the attitude, $\dot{p} = R(\psi) v_{\text{body}}$, so the input gain $R(\psi)$

TABLE II

DRONE BASELINES: IS THE LINEAR MODEL UNDER-PARAMETRIZED, AND WHY NOT A BLACK-BOX MODEL? OPEN-LOOP 15-STEP WORLD-POSITION ERROR AND PER-STEP ROLLOUT COST ($K=512$, $T=15$).

Model	Latent N	Pred. err. (m)	Rollout (ms)
Linear Koopman	13	0.40	0.32
Linear Koopman (large)	29	0.40	0.43
MLP dynamics	—	0.076	1.57
Bilinear (ours)	13	0.002	1.01

The larger linear lift has more parameters and FLOPs than ours yet identical error—linear’s failure is structural. The MLP captures the rotation (far better than linear) but, at matched budget, is less accurate and costlier than the structurally exact bilinear model.

sweeps its full range as the drone yaws. This is the criterion at its sharpest: the bilinear model predicts the world position up to *more than two orders of magnitude* more accurately ($128\times$ at the planning horizon, Fig. 3b). Tracking a figure-8 while yawing along the path (Fig. 3a), ours follows the reference at oracle accuracy (0.06 m mean error, versus 0.06 m for the oracle), whereas the linear model cannot rotate the body command into the world frame and drifts off (1.5 m, a $25\times$ gap). By Remark 1 this gap is structural: the linear model is not under-capacity but unable, in principle, to represent the input–state rotation; the identical deep lift cannot rescue it.

B. Baselines: capacity, same-cost linear, and a black-box model

Two objections to the drone result deserve a direct test: that the linear model is merely under-parametrized, and that one should simply use a generic neural dynamics model. We check both on the drone (Table II). Doubling the linear lift’s latent dimension from $N=13$ to $N=29$ —more parameters and more rollout FLOPs than our bilinear model—leaves its 15-step world-position error unchanged at 0.40 m. Linear’s failure is therefore structural rather than a capacity shortfall, exactly as Remark 1 predicts: no constant input matrix reproduces $R(\psi)$, however large the lift. A nonlinear MLP dynamics model, by contrast, can represent the rotation and is far more accurate than the linear lift (0.076 m), confirming that the coupling, not the dictionary size, is what linear cannot capture. Yet at a matched training budget the MLP remains $\sim 40\times$ less accurate than the bilinear model and costs $1.6\times$ more per rollout: because the drone is control-affine, the bilinear lift is its exact minimal class and reproduces $R(\psi)v$ at matrix–vector cost, whereas the black-box model only approximates it and forgoes the linear-algebra structure the error tube of Prop. 2 relies on. Where the criterion flags the coupling as decisive, the bilinear rollout is thus the more accurate, cheaper, and more analyzable choice.

C. Efficiency where the coupling is weak

On torque-limited pendulum swing-up the input couples to the tracked angle only weakly, and—consistent with the linear baseline [12]—all three controllers swing up and balance. The informative axis here is model capacity. Sweeping the width of the deep dictionary at fixed lifted dimension (Fig. 4),

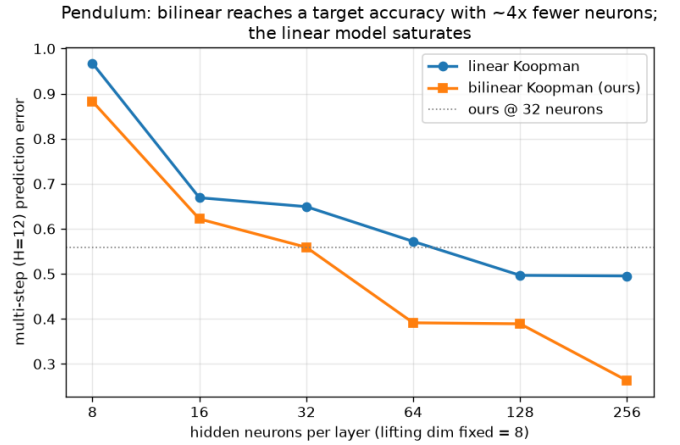


Fig. 4. Pendulum: multi-step prediction error vs. dictionary width, at fixed lifted dimension. The bilinear model reaches a target accuracy with $\approx 4\times$ fewer neurons; the linear model saturates. Both controllers swing up.

the bilinear model reaches a given multi-step accuracy with about $4\times$ fewer neurons. Here bilinearity buys *efficiency* rather than *feasibility*—the same expressiveness at a fraction of the parameters.

D. The boundary: where linear and bilinear coincide

The criterion also says where bilinearity buys nothing: when the input gain barely varies over the operating region. A quadrotor driven by *thrust* is the clean case (Fig. 5): thrust reaches the position only through acceleration, so although the bilinear model is $2\text{--}4\times$ more accurate on the *full* state (it captures the attitude coupling), on the control-relevant *velocity* the linear and bilinear curves coincide. In closed loop both models fly to the goal region; the linear controller in fact lands at oracle accuracy (0.11 versus 0.12 m) while the bilinear one settles slightly farther out (0.43 m), its extra $\{B_i\}$ buying nothing here and only enlarging the training problem (Sec. IV-F)—exactly the regime where the criterion says linear is the right choice. The same holds when the input enters through a configuration-dependent inertia: on a torque-controlled three-link arm (MuJoCo), with a deep lift the two models are effectively indistinguishable in multi-step end-effector prediction (error ratio $0.88\text{--}0.98\times$ across horizons). A cable-driven soft continuum arm (SoftManiSim [21], two and four segments, up to twelve cable inputs) shows the same equivalence in tip prediction (ratio $0.93\text{--}1.09\times$): the smooth, slowly varying curvature keeps the effective input gain nearly constant, so the deep lift alone suffices—consistent with the torque-arm finding that a deep lift closes the gap fixed dictionaries leave to bilinearity [13]. This is precisely the regime of the linear baseline’s own systems [12]—a thrust-driven surface vehicle and an acceleration-commanded quadruped, both with modest heading change—which is why their linear Koopman succeeds; our criterion predicts this, and our boundary systems exhibit the same linear–bilinear equivalence.

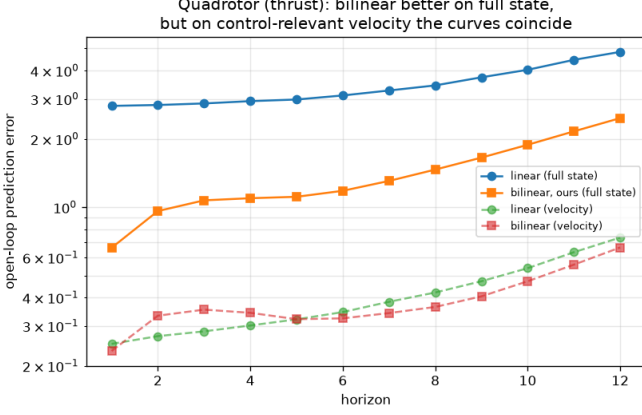


Fig. 5. The boundary. Quadrotor (thrust) open-loop prediction: the bilinear model is 2–4× better on the *full state*, but on the control-relevant *velocity* the linear and bilinear curves coincide—a second-order, weakly varying coupling where a linear lift suffices.

E. Error tube and rollout timing

Tube validity. Theorem 1 and the tube stand on *certified* constants c_x, c_u , which deep dictionaries do not yet supply; we fit them by least squares on the deep model’s residuals and treat them as an empirical proxy. Under this proxy the tube is sound: along held-out rollouts of the deep bilinear model it contains the true lifted error at every step with zero violations over 82,000 steps (Fig. 6). It is, however, conservative: plain least squares yields $\|A\|_2 \approx 1.02$ and $c_x \approx 3.5 \times 10^{-3}$, so the Euclidean tube compounds to $\sim 10^3 \times$ the true error over a long horizon—valid as a certificate, loose as a bound. A tight, contractive tube would require certified, stability-constrained identification (kernel- or SafEDMD-based [18], [19]), the principal next step (Sec. V).

Rollout timing. The lifted rollout is also why a learned model is preferable to the true-dynamics oracle even in simulation. For $K=1024$ samples over $T=30$, one bilinear-Koopman rollout costs 3.5 ms per control step, against 11 ms for a hand-batched integration of the true dynamics (3×) and 72 ms for a physics-engine rollout (20×): the same control rate at a fraction of the compute, the $m+1$ matrix–vector factor over a linear model being negligible against this gap.

F. Ablations and design choices

Three choices double as ablations. *Zero-initialized bilinear terms.* Initializing $\{B_i\}$ to zero makes training start from the linear model and improve monotonically; on the input-rich systems ($m \geq 3$) random initialization yields rollout-unstable models—the failure mode of a naively trained high-input-dimension bilinear lift—whereas for $m \leq 2$ either initialization is stable. *Multi-step training.* One-step regression of either model gives poor long-horizon rollouts; the H -step loss (7), applied identically to both, is what makes either usable inside MPPI. *The error-tube weight β .* Setting $\beta=0$ recovers plain bilinear MPPI and is best for nominal regulation; $\beta>0$ trades a little nominal performance to keep the search in the certified region, paying off when the model is least trustworthy.

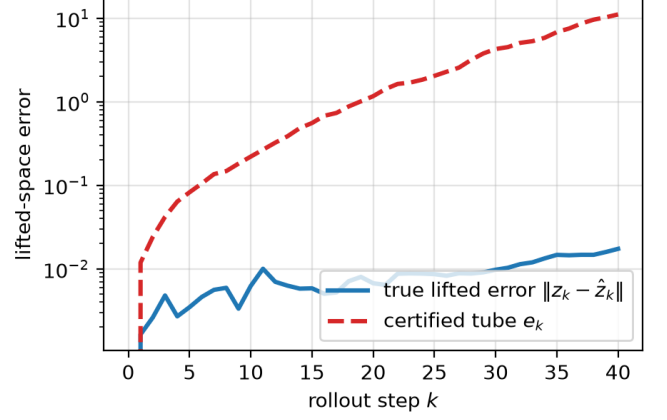


Fig. 6. Error tube e_k (Prop. 2) vs. the true lifted error on a held-out rollout of the *deep bilinear* model, with c_x, c_u fitted as an empirical proxy for certified constants. The tube contains the error at every step (zero violations over 82,000 steps); with $\|A\|_2 \approx 1.02$ it is conservative.

TABLE III
SUMMARY ACROSS SYSTEMS.

System	Input gain	Ours / Oracle / Linear	Remark
Unicycle	$R(\theta)$, max var.	park 95/85/0%	42–48× pred.
Drone (3D)	$R(\psi)$, max var.	0.06/0.06/1.5 m	>100× pred.
Pendulum	weak	all succeed	4× fewer neurons
Quadrotor	2nd-order, low var.	0.43/0.12/0.11 m	vel. pred. coincide
Torque arm	2nd-order (inertia)	—	0.88–0.98× pred.
Soft arm	low var. (curvature)	—	0.93–1.09× pred.

“Input gain” is the variation of $\Phi(x)$ over the operating region (Prop. 1). Unicycle rows report parking rate; drone, mean tracking error; quadrotor, final goal distance. The boundary arms are prediction-only studies (ratio $\approx 1 \times$ means linear and bilinear coincide).

G. Toward hardware validation

[Planned — not yet run.] The simulations isolate the mechanism in a noise-free setting; a hardware study is in preparation to confirm it survives real sensing, actuation, and identification noise. The primary platform is a differential-drive ground robot, whose body-frame velocity commands enter its planar position through the heading—the sharpest real-world instance of the criterion—with a bilinear and a linear deep Koopman model identified from the same on-board odometry logs and no analytical model. We anticipate the simulation gap to persist: the bilinear-model controller should reach goals across a battery of start–goal pairs while the linear-model controller veers off, reproducing Fig. 2 on hardware. Final measurements and videos will be posted to the project page.

V. DISCUSSION AND CONCLUSION

This paper turns a convention into a criterion. Sampling-based control removes the convexity that forced Koopman rollouts to be linear, so the question is no longer which lift is tractable but which lift the system requires; and the answer is a single, testable property—the variation of the input gain

[figure placeholder]

Differential-drive ground robot parking under bilinear vs. linear Koopman MPPI, with motion-capture ground truth. Hardware photograph and overlaid trajectories to be added once the study is run.

Fig. 7. [Planned] Real-robot parking. Anticipated: the bilinear-model controller reaches the goal pose; the linear-model controller veers off, reproducing the simulation result of Fig. 2 on hardware.

$\Phi(x)$ over the operating region (Prop. 1). When that variation is large, the input–state coupling is irreducible: no linear lift, however deep, can represent it (Remark 1), and bilinear becomes mandatory. When it is small, the two coincide. The bilinear model, free to use under sampling, is therefore the right *default*: it is necessary in the first regime, and in the second its only cost is a larger training problem (mitigated by zero initialization, Sec. IV-F); the criterion tells which regime a given system is in.

A map of prior work. This places the literature on one axis. Linear-predictor Koopman MPC [17], [18] and linear Koopman-MPPI [12] operate in the low-variation regime—force- and acceleration-driven systems with modest reorientation—which is exactly why their linear lifts succeed, as our boundary experiments reproduce. Bilinear Koopman realizations [13], [16] were shown advantageous within non-convex MPC and with fixed dictionaries; we add that, under sampling, the bilinear advantage is structural precisely in the high-variation regime and survives a deep lift, while elsewhere a deep lift closes it. The velocity-commanded mobile-base regime where linear must fail—wheeled, legged, and aerial navigation alike—is the one prior linear-Koopman control did not test.

Limitations. Two remain. First, our stability guarantee presumes a certified error bound the deep dictionary does not yet furnish, so the fitted constants of Sec. IV are only an empirical proxy; certified (kernel- or SafEDMD-based) identification, which would also yield the small constants a contractive tube requires, is the immediate next step, together with the hardware study. Second, the criterion presumes a tracked output coupled to the input through smooth kinematics; contact-rich, hybrid dynamics such as legged locomotion at the joint level fall outside it, even where their navigation-level command does not. The takeaway is a default with a known boundary: in sampling-based Koopman control, roll out the bilinear model whenever the input reaches the tracked output through a configuration-dependent gain that varies—and know, in advance, when it does.

REFERENCES

- [1] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Aggressive driving with model predictive path integral control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2016, pp. 1433–1440.
- [2] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic MPC for model-based reinforcement learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 1714–1721.
- [3] G. Williams, A. Aldrich, and E. A. Theodorou, “Model predictive path integral control: From theory to parallel computation,” *J. Guid. Control Dyn.*, vol. 40, no. 2, pp. 344–357, 2017.
- [4] H. J. Kappen, “Path integrals and symmetry breaking for optimal control theory,” *J. Stat. Mech.: Theory Exp.*, p. P11011, 2005.
- [5] E. A. Theodorou, J. Buchli, and S. Schaal, “A generalized path integral control approach to reinforcement learning,” *J. Mach. Learn. Res.*, vol. 11, pp. 3137–3181, 2010.
- [6] L. Park, K. Jang, and S. Kim, “A systematic survey of model predictive path integral control: From stochastic theory to real-time robotics,” 2026, under review, ISA Transactions.
- [7] B. O. Koopman, “Hamiltonian systems and transformation in Hilbert space,” *Proc. Natl. Acad. Sci.*, vol. 17, no. 5, pp. 315–318, 1931.
- [8] I. Mezić, “Spectral properties of dynamical systems, model reduction and decompositions,” *Nonlinear Dyn.*, vol. 41, pp. 309–325, 2005.
- [9] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, “A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition,” *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [10] J. L. Proctor, S. L. Brunton, and J. N. Kutz, “Dynamic mode decomposition with control,” *SIAM J. Appl. Dyn. Syst.*, vol. 15, no. 1, pp. 142–161, 2016.
- [11] B. Lusch, J. N. Kutz, and S. L. Brunton, “Deep learning for universal linear embeddings of nonlinear dynamics,” *Nat. Commun.*, vol. 9, p. 4950, 2018.
- [12] W. Hao, Y. Fang, Z. Lu, and S. Mou, “Accelerating sampling-based control via learned linear Koopman dynamics,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2026, arXiv:2603.05385.
- [13] D. Bruder, X. Fu, and R. Vasudevan, “Advantages of bilinear Koopman realizations for the modeling and control of systems with unknown dynamics,” *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4369–4376, 2021.
- [14] C. Folkestad and J. W. Burdick, “Koopman NMPC: Koopman-based learning and nonlinear model predictive control of control-affine systems,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021, pp. 7350–7356.
- [15] G. Mamakoukas, M. L. Castano, X. Tan, and T. D. Murphey, “Local Koopman operators for data-driven control of robotic systems,” in *Robotics: Science and Systems (RSS)*, 2019.
- [16] L. C. Jacob, R. Tóth, and M. Schoukens, “Koopman form of nonlinear systems with inputs,” *Automatica*, vol. 162, p. 111525, 2024.
- [17] M. Korda and I. Mezić, “Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control,” *Automatica*, vol. 93, pp. 149–160, 2018.
- [18] R. Strässer, M. Schaller, K. Worthmann, J. Berberich, and F. Allgöwer, “An overview of Koopman-based control: From error bounds to closed-loop guarantees,” *Annual Reviews in Control*, vol. 61, p. 101035, 2026.
- [19] —, “SafEDMD: A certified learning architecture tailored to data-driven control of nonlinear dynamical systems,” arXiv:2411.10359, 2024.
- [20] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.
- [21] M. Kasaei, H. Kasaei, and M. Khadem, “SoftManiSim: A fast simulation framework for multi-segment continuum manipulators tailored for robot learning,” in *Proc. Conf. Robot Learning (CoRL)*, 2024.